Iterative Attractor Dynamics for Classification: A Dynamical Classification Head with Anchor-Guided Basin Settling

Chetan Patil
chetan12patil@gmail.com
https://github.com/chetanxpatil/livnium

March 2026

Abstract

We present Livnium, a classification head implemented as a discrete-time dynamical system. Instead of projecting an encoded representation to logits via a single linear layer, Livnium evolves the hidden state through L steps of geometry-aware updates under learned anchor forces before readout. Three anchor vectors define label basins in a shared embedding space; the state vector $\boldsymbol{h}$ moves under cosine-based divergence forces until it settles near one basin, and the hidden state is iteratively refined under anchor-guided forces before final readout. We define a Lyapunov function $V(\boldsymbol{h}) = (\tau - \cos(\boldsymbol{h}, A_y))^2$ and show that, in the absence of the learned residual, the dynamics are locally contracting (a first-order local descent argument, not a global stability proof). The approach is evaluated on the Stanford Natural Language Inference (SNLI) benchmark, achieving 76% dev accuracy with a frozen bag-of-words encoder. We report the geometric properties of the force field, including a measured 135.2° mismatch between the implemented Euclidean radial forces and the true cosine gradient, and discuss implications for convergence. Code and pretrained weights are publicly available.

1 Introduction

The standard neural network classifier computes $y = \text{softmax}(Wh + b)$: a single affine transformation from a learned representation h to class logits. This is a static map — the label is determined by a single projection of the representation.

We explore an alternative: what if the classification head were a small dynamical system that *evolves* the representation through several steps before producing a label? Instead of computing the label from the raw encoding h_0, we let the state evolve:

$$h_0 \;\to\; h_1 \;\to\; \cdots \;\to\; h_L \;\to\; \text{logits}$$

where each update step applies learned residuals and geometric forces from label-specific anchor vectors. The final state h_L is the result of a settling process, not a one-shot projection.

This idea has historical roots. Hopfield networks [Hopfield, 1982] used energy minimisation for content-addressable memory. Modern equilibrium models [Bai et al., 2019] compute representations as fixed points of implicit layers. Our approach sits between the two: it uses an explicit, finite-step dynamical rule with interpretable geometric structure, rather than an implicit fixed-point iteration.

The contribution of this paper is:

1. A concrete update rule for attractor-based classification with three label basins and cosine-ring equilibria (Section 2).
2. A Lyapunov analysis showing local contraction of the base dynamics (Section 3).
3. An empirical characterisation of the geometric inconsistency between the implemented forces and true cosine gradients (Section 4).
4. SNLI results with analysis of per-class behavior and inference speed (Section 5).

We emphasize that this work replaces only the classification head. The encoder (a bag-of-words mean pooling in our experiments) is orthogonal to the attractor dynamics. The question we address is: *does iterative basin settling provide useful inductive bias relative to a standard readout when the encoder is held fixed?*

2 Method

2.1 Setup

Given a premise p and hypothesis q, an encoder produces sentence vectors $\boldsymbol{v}_p, \boldsymbol{v}_q \in \mathbb{R}^d$. The initial state for the collapse dynamics is:

$$h_0 = \boldsymbol{v}_q - \boldsymbol{v}_p + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.01^2 I)$$

The difference vector captures the directional relationship between premise and hypothesis. The small noise ϵ breaks symmetry in the attractor landscape.

2.2 Label Geometry

Three learned anchor vectors $A_E, A_C, A_N \in \mathbb{R}^d$ (entailment, contradiction, neutral) define the geometry of the label space. Each anchor is unit-normalised during the forward pass. The *equilibrium* for each label is not the anchor point itself but a **cosine-similarity ring**:

$$\cos(h, A_y) = \tau$$

where $\tau = 0.38$ is a fixed barrier constant. This defines a $(d-2)$-dimensional manifold on the unit hypersphere — a "ring" at a fixed angular distance from each anchor.

2.3 Update Rule

At each step $t = 0, \ldots, L-1$:

$$h_{t+1} = h_t + \delta_\theta(h_t) - s_y \cdot D(h_t, A_y) \cdot \hat{\boldsymbol{n}}(h_t, A_y) - \beta \cdot B(h_t) \cdot \hat{\boldsymbol{n}}(h_t, A_N) \tag{1}$$

where:

$$D(h, A) = \tau - \cos(h, A) \tag{2}$$

$$\hat{\boldsymbol{n}}(h, A) = \frac{h - A}{\|h - A\|} \tag{3}$$

$$B(h) = 1 - |\cos(h, A_E) - \cos(h, A_C)| \tag{4}$$

The terms in (1) are:

- $\delta_\theta(h_t)$: a learned residual (2-layer MLP with Tanh).
- $s_y \cdot D \cdot \hat{\boldsymbol{n}}$: the **anchor force**. When $\cos(h, A_y) < \tau$, divergence is positive and the force pulls h toward the anchor. When $\cos(h, A_y) > \tau$, the force pushes h away. The equilibrium ring acts as an attractor, not the anchor itself.
- $\beta \cdot B(h) \cdot \hat{\boldsymbol{n}}(h, A_N)$: the **neutral boundary force**. $B(h)$ measures equidistance from A_E and A_C; when h sits on the E–C boundary, $B \approx 1$ and the force pulls h toward neutral territory.

During training, only the anchor corresponding to the correct label y exerts force ($s_y > 0$; other strengths are zero). During inference, all three anchors act simultaneously with fixed hyperparameter strengths ($s_E = s_C = 0.1$, $s_N = 0.05$; these are the same values used during training, not learned or adapted), and the label is determined by which basin the state settles nearest.

A soft norm cap prevents unbounded growth:

$$h \leftarrow \begin{cases} h \cdot \frac{10}{\|h\|+10^{-8}} & \text{if } \|h\| > 10 \\ h & \text{otherwise} \end{cases}$$

2.4 Classification Head

The full classification pipeline is: encoder → attractor dynamics → readout network. After L collapse steps, the refined state h_L is passed to a two-layer readout MLP along with auxiliary geometric features: cosine alignment between $\boldsymbol{v}_p$ and $\boldsymbol{v}_q$, opposition signal $\cos(-\boldsymbol{v}_p, \boldsymbol{v}_q)$, Euclidean distance $\|\boldsymbol{v}_q - \boldsymbol{v}_p\|$, norm features, and a learned neutral alignment signal. The readout MLP outputs logits for the three SNLI classes. We note that the final label decision is made by this readout network, not purely by basin membership; the attractor dynamics refine the representation that the readout network receives.

2.5 Dynamic Basin Field

During training, a **basin field** maintains multiple micro-basins per label. Each basin has a center vector updated via exponential moving average on the unit sphere. New basins are spawned when tension (divergence magnitude) stays high and alignment is low, indicating an underserved region of the label manifold. Basins are periodically pruned and merged.

At inference, the basin field is read-only: the system routes each sample to the nearest basin using label-free cosine similarity. No ground-truth labels are used during inference routing.

3 Lyapunov Analysis

Define the candidate Lyapunov function:

$$V(h) = \big(\tau - \cos(h, A_y)\big)^2 = D(h, A_y)^2 \tag{5}$$

$V = 0$ exactly on the equilibrium ring $\cos(h, A_y) = \tau$. V serves as a local diagnostic measuring distance from the target ring, not a global energy function for the full system.

Proposition 1 (Local contraction without learned residual)**.** *When $\delta_\theta = 0$ and only the correct anchor force acts, $V(h_{t+1}) \leq V(h_t)$ for sufficiently small force strength s_y. This is a first-order local descent argument, not a global stability proof.*

Proof sketch. The anchor force component is $-s_y \cdot D(h, A) \cdot \hat{\boldsymbol{n}}(h, A)$. The change in cosine alignment due to this force involves:

$$\nabla_h \cos(h, A) \cdot \hat{\boldsymbol{n}}(h, A) = -\frac{r \sin^2 \theta}{\alpha \|h - A\|}$$

where $\alpha = \|h\|$, $r = \|A\|$, and θ is the angle between h and A. Since $\sin^2 \theta \geq 0$ and all norms are positive, the dot product is non-positive. Therefore the anchor force moves $\cos(h, A)$ toward τ (reduces $|D|$), which reduces V. □

Empirical verification (5,000 samples, δ_θ scale varied):

δ_θ scale	$V(h_{t+1}) \leq V(h_t)$	mean ΔV
0.001	100.0%	−0.00131
0.01	99.3%	−0.00112
0.05	70.9%	−0.00040
0.10	61.3%	+0.00001

Table 1: Lyapunov descent rate as the learned residual strength increases. The base dynamics are contracting; the MLP residual can partially oppose contraction at larger scales.

Limitation. Global convergence is not proven. The learned residual δ_θ can dominate the anchor force at large scales, and finite step sizes can overshoot the attractor ring. The Lyapunov guarantee is local and conditional on δ_θ being small relative to the anchor force.

4 Geometric Inconsistency

A notable property of the update rule is that the force magnitude is cosine-based (Eq. 2) but the force direction is Euclidean radial (Eq. 3). The true gradient of cosine similarity with respect to h is tangential on the hypersphere, not radial.

To quantify this, we measured the angle between the implemented force direction $\hat{\boldsymbol{n}}(h, A)$ and the true cosine gradient direction $\nabla_h \cos(h, A)/\|\nabla_h \cos(h, A)\|$ across 1,000 random samples in $\mathbb{R}^{256}$:

$$\text{mean angle} = 135.2^\circ \pm 2.5^\circ \tag{6}$$

This means the system applies forces that are roughly orthogonal-plus-opposing to the true gradient. In physical terms, this is a **non-conservative force field** — there is no scalar potential whose gradient produces these forces.

Despite this, the system converges empirically. We interpret the anchor force as a discrete-time attractor rule that happens to use cosine as a distance proxy, not as gradient descent on an energy surface. The Euclidean direction provides a consistent "pull toward the anchor" that, combined with the cosine-based magnitude, produces effective basin capture even though the geometry is formally inconsistent.

This inconsistency may even be beneficial: the non-conservative "sideways" component could act as implicit regularisation, preventing the state from collapsing directly onto the anchor and instead encouraging exploration of the attractor ring.

5 Experiments

5.1 Dataset and Encoder

We evaluate on the Stanford Natural Language Inference (SNLI) dataset [Bowman et al., 2015], which contains approximately 570k sentence pairs labeled as entailment, contradiction, or neutral.

The encoder is a pretrained bag-of-words model: a learned embedding table (trained on WikiText-103 [Merity et al., 2016]) followed by mean pooling over tokens. This produces 256-dimensional sentence vectors. The encoder is frozen during attractor head training.

5.2 Configuration

Parameter	Value
Embedding dimension	256
Collapse steps L	6
Barrier constant τ	0.38
Anchor strengths s_E, s_C	0.1
Anchor strength s_N	0.05
Neutral boundary boost β	0.05
Total parameters	$\sim$2M

Table 2: Model configuration.

5.3 Results

Class	Dev Accuracy
Overall	**76.00%**
Entailment	80.6%
Contradiction	75.2%
Neutral	72.2%

Table 3: SNLI dev set accuracy (label-free inference routing).

For context, bag-of-words baselines typically achieve $\sim$80% on SNLI, decomposable attention [Parikh et al., 2016] reaches $\sim$86%, and fine-tuned BERT [Devlin et al., 2019] achieves $\sim$90%.

Our result is below the bag-of-words baseline. However, the attractor head receives mean-pooled bag-of-words vectors, which are known to lose word order. Sentences such as "a dog bites a man" and "a man bites a dog" produce nearly identical encodings, creating an encoder-side ceiling that is difficult to separate from head-side limitations. A controlled comparison with a frozen BERT encoder would isolate the attractor head's contribution.

5.4 Inference Speed

5.5 Neutral Class Analysis

Neutral is the weakest class (72.2% vs. 80.6% for entailment). The confusion matrix shows that neutral samples are frequently misclassified as both entailment and contradiction. The neutral

Component	ms / batch (32)	Samples / sec
Livnium attractor head	0.4	85,335

Table 4: Head-only inference speed on CPU. This measures only the attractor dynamics and readout, excluding all encoding time. It is **not** an end-to-end comparison: a fair speed comparison would need to include the encoder for both systems.

boundary force (Eq. 4) was specifically designed to address this, pulling states toward neutral when they are equidistant from the E and C anchors. Without it, neutral accuracy drops further.

The neutral class is intrinsically harder because it is defined by the *absence* of a strong entailment or contradiction signal. With a bag-of-words encoder that loses word order, many genuinely neutral pairs produce embeddings that overlap with entailment or contradiction pairs.

6 Related Work

Hopfield networks [Hopfield, 1982] pioneered energy-based attractor dynamics for memory retrieval. Modern Hopfield networks [Ramsauer et al., 2020] extend this to continuous states with exponential storage capacity.

Deep equilibrium models (DEQ) [Bai et al., 2019] compute representations as fixed points of an implicit layer, using root-finding rather than explicit unrolling. Livnium uses explicit L-step dynamics with no fixed-point solver.

Neural ODEs [Chen et al., 2018] parameterise continuous-time dynamics. Livnium operates in discrete time with interpretable geometric forces rather than a learned vector field.

Prototype networks [Snell et al., 2017] classify by distance to class prototypes. Livnium's anchors serve a similar role, but the state is iteratively refined rather than directly compared.

Natural language inference baselines range from bag-of-words [Bowman et al., 2015] to decomposable attention [Parikh et al., 2016] to fine-tuned transformers [Devlin et al., 2019]. Our work is orthogonal to the encoder and addresses only the classification head.

7 Limitations and Future Work

1. **Train/inference mismatch.** During training, only the correct-label anchor exerts force; during inference, all three anchors act simultaneously. This means the dynamics the model learns are not identical to the dynamics used at test time. The mismatch may limit settling accuracy and is a structural gap that future work should address.

2. **Encoder ceiling.** The bag-of-words encoder cannot distinguish word order, which limits accuracy regardless of the classification head. A fair evaluation requires a controlled comparison: frozen BERT encoder with attractor head vs. frozen BERT encoder with linear head.

3. **No global convergence.** The Lyapunov guarantee is local and conditional on the learned residual being small. Finite step sizes and the learned MLP can cause the state to escape basins.

4. **Geometric inconsistency.** The 135° mismatch between Euclidean radial forces and true cosine gradients is acknowledged but not resolved. Whether replacing the direction with the true tangential gradient would improve or harm performance is an open question.

5. **Limited benchmarking.** Results are reported on SNLI only. Evaluation on MultiNLI, adversarial NLI, and out-of-distribution robustness tasks would better characterise the approach.

6. **Hyperparameter sensitivity.** The barrier constant $\tau = 0.38$, force strengths, and number of collapse steps were tuned manually. A principled selection method is needed.

8 Conclusion

We have presented a classification head that operates as an iterative dynamical system rather than a static projection. The core idea — replacing $h \to Wh + b$ with $h_0 \to h_1 \to \cdots \to h_L$ under geometric anchor forces — is simple and fast, and comes with a partial stability guarantee via a Lyapunov function.

The current results do not exceed standard baselines on SNLI. However, the approach opens a different design axis: classification heads with built-in iterative refinement, interpretable trajectory dynamics, and geometric stability properties. Whether this axis leads to practical gains remains to be demonstrated with stronger encoders and broader evaluation.

Code: https://github.com/chetanxpatil/livnium
Model: https://huggingface.co/chetanxpatil/livnium-snli

References

S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *NeurIPS*, 2019.

S. Bowman, G. Angeli, C. Potts, and C. Manning. A large annotated corpus for learning natural language inference. In *EMNLP*, 2015.

R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT*, 2019.

J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *PNAS*, 79(8):2554–2558, 1982.

S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. *arXiv:1609.07843*, 2016.

A. Parikh, O. Täckström, D. Das, and J. Uszkoreit. A decomposable attention model for natural language inference. In *EMNLP*, 2016.

H. Ramsauer, B. Schäfl, J. Lehner, P. Seidl, M. Widrich, T. Adler, L. Gruber, M. Holzleitner, M. Pavlović, G. Sandve, V. Greiff, D. Kreil, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter. Hopfield networks is all you need. In *ICLR*, 2021.

J. Snell, K. Swersky, and R. Zemel. Prototypical networks for few-shot learning. In *NeurIPS*, 2017.